

Razorpay Integration — Frontend Requirements

Project: urbancompany (Angular 17)
Prepared by: Frontend team
Date: 2026-10-01

1. Purpose

This document lists what the **frontend** needs to integrate Razorpay checkout: the keys and IDs, the APIs we need from the backend, the customer details we collect, and the basic screens to build after a successful payment.

Important: The frontend only ever uses the **Key ID**. The **Key Secret** and **Webhook Secret** stay on the backend and must never be shared with the frontend or added to Angular code.

2. Keys & IDs required for the frontend

2.1 From the client (Razorpay Dashboard → Account & Settings → API Keys)

#	Item	Example	Used for
1	Test Key ID	rzp_test_xxxxxxxxxxx	Development and testing
2	Live Key ID	rzp_live_xxxxxxxxxxx	Production (after Razorpay account activation)
3	Business display name	Urban Company	Title shown in the payment popup
4	Logo	Square PNG, min 256 × 256 px	Shown in the payment popup
5	Theme colour	#6E42E5	Popup colour, to match our site
6	Payment methods to enable	UPI, Cards, Net Banking, Wallets	Enabled by the client in the dashboard

The Key ID can also be returned by the backend in the create-order response, so the frontend does not need to store it at all. Either option is fine — to be agreed with the backend team.

2.2 From the backend (per payment)

ID	Comes from	Used for
order_id (order_XXXXXXX)	Backend create-order API	Required to open the Razorpay popup
amount (in paise)	Backend create-order API	Shown in popup — e.g. ₹499 = 49900
currency	Backend create-order API	INR
bookingId	Backend create-order API	Our own booking reference

2.3 Returned by Razorpay to the frontend after payment

On success, Razorpay returns three values to the frontend. **We send all three to the backend for verification** — the frontend must not mark a booking as paid by itself.

Field	Example
razorpay_payment_id	pay_xxxxxxxxxxx
razorpay_order_id	order_xxxxxxxxxxx
razorpay_signature	long hex string

On failure, Razorpay returns an error `code`, `description` and `reason`, which we show to the user.

3. APIs required from the backend

#	Method & Endpoint	Request (from frontend)	Response (to frontend)
1	POST /api/payments/create-order	Cart items (service ID + qty), tip, address, slot, customer details	order_id, amount, currency, bookingId, key_id (optional)
2	POST /api/payments/verify	razorpay_payment_id, razorpay_order_id, razorpay_signature, bookingId	success, booking details
3	GET /api/bookings	— (logged-in user)	List of the user's bookings
4	GET /api/bookings/:id	—	Booking details + current status
5	POST /api/bookings/:id/cancel	Reason (optional)	Updated status, refund status

The backend must calculate the final amount from its own price list. The cart is stored in the browser, so prices sent from the frontend cannot be trusted.

4. Customer details collected on the frontend

These are sent with the create-order request and used to pre-fill the Razorpay popup.

Field	Required	Status in current app
Mobile number	Yes	✅ Available from login
Name	Yes	⚠️ Currently hard-coded — needs an input
Email	Optional	❌ Not collected — add field
Service address (house no., street, landmark, city, pincode)	Yes	❌ Hard-coded — address form needed
Date & time slot	Yes	❌ Slot selection needed
Cart items & quantity	Yes	✅ Available
Tip	Optional	✅ Available
Accept T&C / cancellation policy	Yes	❌ Add "By proceeding you agree..." line

5. Payment flow (frontend view)

1. User clicks "Pay"
2. Frontend → POST /api/payments/create-order → gets order_id, amount
3. Frontend opens Razorpay popup with key_id + order_id + prefill (name, phone, email)
4. User pays (UPI / Card / Net Banking)
5. Razorpay → returns payment_id, order_id, signature to frontend
6. Frontend → POST /api/payments/verify → backend confirms
7. On success: clear cart, go to Booking Confirmation page
On failure / popup closed: keep cart, show message, allow retry

6. Frontend tasks

#	Task	File
1	Load Razorpay script <code>https://checkout.razorpay.com/v1/checkout.js</code>	<code>src/index.html</code> or loaded on demand
2	Store Test / Live Key ID per environment (if not returned by API)	<code>src/environments/</code> (new)
3	Add payment & booking API URLs	<code>src/app/constants/urls.ts</code>
4	Create <code>PaymentService</code> (create order, open popup, verify)	<code>src/app/services/payment.service.ts</code> (new)
5	Update <code>pay()</code> — replace current instant success with real flow	<code>src/app/modules/checkout/checkout.component.ts</code>
6	Show loader and disable Pay button while processing	<code>checkout.component.html</code>
7	Handle payment failure and popup close	<code>checkout.component.ts</code>
8	Add name, email, address and slot inputs	Checkout page

7. Modules after successful payment (basic)

#	Module	What it shows
1	Booking Confirmation	Booking ID, payment ID, service, date & slot, address, amount paid (update existing <code>booking-success</code> page)
2	My Bookings	List of upcoming and past bookings with status (inside Profile)
3	Booking Details / Order Tracking	Status steps: Confirmed → Professional Assigned → On the way → Completed
4	Cancel Booking	Cancel option with refund status (Initiated / Refunded)
5	Payment Failed & Retry	Failure message with a "Try again" button

8. Testing (Razorpay Test Mode)

Case	How to test	Expected
UPI success	UPI ID <code>success@razorpay</code>	Booking confirmed
UPI failure	UPI ID <code>failure@razorpay</code>	Error shown, cart kept
Card payment	Test cards from Razorpay docs	Booking confirmed
Popup closed	Close the popup	Message shown, cart kept
Double click	Click Pay twice	Only one order created
Mobile view	Pay on a phone screen	Popup fits screen

9. Checklist — information needed

From client

- ☐ Test Key ID
- ☐ Live Key ID (after account activation)
- ☐ Logo, display name, theme colour
- ☐ Payment methods to enable
- ☐ Cancellation & refund policy text

From backend team

- ☐ Create-order API
- ☐ Verify-payment API
- ☐ Bookings list, details and cancel APIs
- ☐ Booking status values and their meaning